# Morphological Processing

Week 7 — Session 1: Erosion, dilation, opening & closing

Practical Image Processing

Lecture 7.1 ▪ 60–70 min

# Recap & today

- Wk2/8: thresholding gives a **binary** image — but it is messy (specks, gaps, ragged edges).
- Today: **morphology** — operations that grow, shrink, and clean binary shapes with a **structuring element**.
- Session 1 builds the four workhorses: **erosion**, **dilation**, **opening**, **closing**.

**DocScan milestone**

After thresholding the meter digits, morphology removes noise and fills gaps so each digit is a clean, separable blob.

# Roadmap

The structuring element

Erosion

Dilation

Opening & closing

In-class exercise

Wrap-up

# The structuring element

# From smoothing to shape reasoning

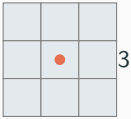
- Blurring *averages* pixels; morphology *compares* the shape of a small probe to the image.
- It works on a **binary** image (foreground = white = 1, background = black = 0).
- Every operation is built from two questions asked at each pixel: does the probe *fit*, or does it *hit*?

**Key idea**

Morphology is shape surgery: it reasons about geometry, not brightness.

# The structuring element (SE)

- A small shape (the **SE** / kernel) that slides over the image — a probe with a chosen **anchor**.
- Common shapes: rectangle, ellipse, cross. Its **size** sets how aggressive the effect is.
- The anchor (usually the centre) is the pixel that receives the result.



3×3 box

anchor = centre

# Building an SE in OpenCV

```python
import cv2
rect  = cv2.getStructuringElement(cv2.MORPH_RECT,    (3, 3))
ell   = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
cross = cv2.getStructuringElement(cv2.MORPH_CROSS,   (3, 3))
print(cross)   # 1s form a plus sign; 0s are ignored
```

- Only the `1` cells of the SE take part; the `0` cells are ignored.
- Odd sizes $(3, 5, 7, \dots)$ keep a well-defined centre anchor.

# Erosion

# Erosion — the “fit” rule

- Keep a foreground pixel *only if* the SE fits *entirely* inside the shape at that position.
- Effect: shapes **shrink**, thin protrusions vanish, and touching objects can **separate**.
- Formally $A \ominus B = \{z : B_z \subseteq A\}$ — the set of anchor positions where the shifted SE $B_z$ stays inside $A$.

**Math**

With a $3 \times 3$ box SE, a foreground pixel survives only if all 8 neighbours are also foreground.

# Erosion — worked 1D-ish example

Foreground $= 1$. Erode with a $1 \times 3$ horizontal SE (a pixel survives only if its left *and* right neighbours are 1).

input row

| 0 | 1 | 1 | 1 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|

after erosion

| 0 | 0 | 1 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|

- The lone 1 dies (no neighbours); the run of three shrinks to its centre.
- Erosion removes objects thinner than the SE — that is how it deletes noise.

# Erosion & dilation on real digits



binary digits



erosion (thinner)



dilation (thicker)

- **Erosion**: strokes get thinner; hairline specks disappear entirely.
- **Dilation**: strokes get thicker; small gaps close up.

# Dilation

# Dilation — the “hit” rule

- Set a pixel to foreground if the SE *touches* (hits) the shape anywhere at that position.
- Effect: shapes **grow**, small holes and gaps fill, nearby pieces **connect**.
- Formally $A \oplus B = \{z : B_z \cap A \neq \varnothing\}$ — anchor positions where the shifted SE overlaps $A$.

**Math**

With a $3 \times 3$ box SE, a background pixel becomes foreground if *any* of its 8 neighbours is foreground.

# Dilation — worked example

Dilate the same row with the $1 \times 3$ SE (a pixel becomes 1 if it *or* a neighbour is 1).

input row

| 0 | 1 | 1 | 1 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|

after dilation

| 1 | 1 | 1 | 1 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|

- Each run grows by one cell on each side; the one-cell gap between the runs is bridged.
- Dilation fills gaps smaller than the SE — that is how it repairs broken strokes.

# Iterations vs a bigger SE

- Two ways to make erosion/dilation stronger: a *larger* SE, or *more iterations* of a small SE.
- `iterations=n` applies the same SE $n$ times — cheap, and easy to tune interactively.
- Rough equivalence: $n$ passes of a $3 \times 3$ box $\approx$ one $(2n{+}1) \times (2n{+}1)$ box.

**Math**

Reach of erosion $\approx \lfloor k/2 \rfloor$ pixels for a $k \times k$ SE (per pass): a $5 \times 5$ box eats 2 pixels off every edge.

# Erosion and dilation are duals

- Dilating the foreground = eroding the background (and vice versa): $(A \ominus B)^c = A^c \oplus \hat{B}$.
- **Erosion** removes: thin protrusions, small objects; it *separates* touching objects.
- **Dilation** adds: fills gaps, connects nearby pieces, *thickens* thin strokes.

**✕ Pitfall**

Each also distorts: erosion can break a thin digit; dilation can merge two close digits. Size matters — choose the smallest SE that works.

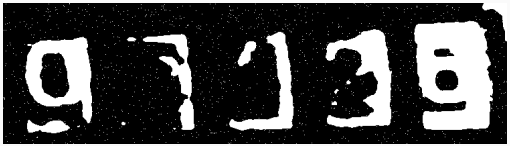
# Neither preserves size — so we compose them




- Raw erosion or dilation changes the object's size — often undesirable.
- Doing one then the *other* (next section) cleans the shape while restoring its size.

# Opening & closing

# Opening = erode then dilate



binary + speckles



opening: specks gone

- **Opening** removes small bright specks while keeping the digits' size: erosion kills the specks, dilation restores what remains.
- The classic "clean up salt noise on a binary mask" operation.
- $A \circ B = (A \ominus B) \oplus B$.

# Why opening keeps size but kills specks

- A speck smaller than the SE is erased by the erosion and never comes back in the dilation.
- A digit larger than the SE is shrunk by erosion, then re-grown by dilation to (about) its original size.
- Geometric view: opening keeps exactly the parts where the SE could *fully fit*, rolled along the shape.

**✔ Good practice**

Opening is **idempotent**: applying it twice does nothing more than once. One well-sized pass is enough.

# Closing = dilate then erode



binary



closing: gaps filled

- **Closing** fills small holes and gaps inside shapes while keeping the size.
- Repairs a digit broken by thresholding into a solid stroke.
- $A \bullet B = (A \oplus B) \ominus B$.

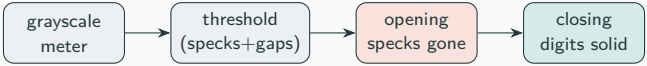
# Closing — worked example

Close the broken row with the $1 \times 3$ SE (dilate, then erode).

| | | | | | |
|---|---|---|---|---|---|
| input | 1 | 1 | 0 | 1 | 1 |
| dilate | 1 | 1 | 1 | 1 | 1 |
| then erode | 1 | 1 | 1 | 1 | 1 |

- The one-pixel gap is bridged by the dilation and *survives* the erosion — the stroke is repaired.
- A gap wider than the SE would *not* close: the SE size sets the maximum repairable gap.

# The water-meter, step by step




- After thresholding, the “7” has a hairline break and a few salt specks float nearby.
- Opening deletes the specks; closing welds the “7” back into one stroke — now countable.

**✓ Good practice**

This open-then-close pair is the core of the DocScan digit cleanup you will ship this week.

# Opening vs closing: remember the effect

| Operation | Order | Effect |
|---|---|---|
| Opening | erode $\rightarrow$ dilate | removes small *bright* specks, breaks thin links |
| Closing | dilate $\rightarrow$ erode | fills small *dark* holes, bridges gaps |

**Key idea**

Opening cleans the outside; closing repairs the inside. Both keep the overall size, unlike raw erosion/dilation.

# In-class exercise

# In-class exercise (10 min)

**Pick the operation**

For each defect on your thresholded meter, name the single morphology operation that fixes it — and the SE size you would start with.

1. Scattered single white pixels (salt) around the digits.
2. One digit is broken into two pieces by a threshold gap.
3. Two adjacent digits are touching and need to be separated for counting.
4. Trace the "$0, 1, 1, 0, 1, 1, 1, 0$" row through erosion *and* dilation with a $1 \times 3$ SE.

# Debrief

- (1) Salt specks $\rightarrow$ **opening** with a small $(3 \times 3)$ SE.
- (2) Broken digit $\rightarrow$ **closing** sized just above the gap width.
- (3) Touching digits $\rightarrow$ careful **erosion** to separate (or watershed, Session 2 / Week 8).
- (4) Erode $\rightarrow 0, 0, 1, 0, 0, 1, 0, 0$; dilate $\rightarrow 1, 1, 1, 1, 1, 1, 1, 1$.

**Key idea**

Diagnose the defect (specks / gaps / merges), then choose the matching operation and the smallest SE that fixes it.

# Wrap-up

# Common pitfalls

**✗ Oversized SE**

A too-large SE turns digits into unrecognizable blobs, or fuses neighbours. Tune the size on real data.

**✗ Wrong polarity**

On a *white-on-black* mask, opening removes bright specks; if your foreground is inverted, open and close swap roles. Check which pixels are "1".

# The four operations in code

```
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
er = cv2.erode(binary, k, iterations=1)          # shrink
di = cv2.dilate(binary, k, iterations=1)         # grow
op = cv2.morphologyEx(binary, cv2.MORPH_OPEN,  k)  # remove specks
cl = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, k)  # fill gaps
```

`morphologyEx` wraps open/close (and more, next session) in one call.

# What you should now be able to do

- Explain erosion/dilation via a structuring element (fit vs hit), and trace them by hand.
- Predict how the SE size changes the strength of each effect.
- Use opening (remove specks) and closing (fill gaps), and say why each preserves size.

# Next session

**Session 2 preview**

We build on erosion/dilation to get **gradient**, **top-hat**, and **black-hat**; extend morphology to **grayscale**; remove ruled lines while keeping text; and assemble the full DocScan cleanup pipeline.

- Bring your thresholded meter image — we will clean it end-to-end.

## Threshold makes it binary; erosion and dilation reshape it.

Opening removes specks, closing fills gaps — and both keep the size.